



Jain Deemed to be University

MSc Data Science & Analytics

BIG DATA ANALYTICS CAPSTONE PROJECT

RETAIL DEMAND FORECASTING

Big Data Analytics Pipeline

Final Technical Documentation

Participants

Participant Name	USN
Aisha Rasifa PS	25MSDSR004
Abina A	25MSDSR003
Anuska Misra	25MSDSR014
Akhila Vaidya	25MSDSR006
Divyanshi Mishra	25MSDSR0016

Project GitHub Repository

<https://github.com/divyanshi2323/Retail-Demand-Forecasting-Big-Data>

RETAIL DEMAND FORECASTING

Big Data Analytics Pipeline — Final Technical Documentation

University: Jain Deemed to be University	Owner: Student 5 – Integration + Documentation
Programme: MSc Data Science & Analytics	Dataset Scope: 76,000 records × 16 columns
Project Type: Big Data Analytics Capstone	Tech Stack: HDFS, Hive, Spark, MLlib, HBase

1. Project Overview

This project delivers an end-to-end Big Data Analytics pipeline designed to process, model, analyze, and forecast retail product demand at a granular store-product-day level. Utilizing a distributed computing stack—comprising HDFS for raw storage, Apache Hive for structured SQL data modelling, Apache Spark / PySpark for distributed transformations and machine learning, and Apache HBase for low-latency summary lookups—the system translates 76,000 historical records into actionable business intelligence.

1.1 Project Title

Retail Demand Forecasting – Big Data Analytics Pipeline

1.2 Team Structure and Primary Responsibilities

Role	Student Name	Primary Technical Responsibility
Student 1	Aisha Rasifa PS	Data Ingestion + HDFS Storage Architecture
Student 2	Abina A	Hive Data Modelling + Schema Definition + SQL Analytics
Student 3	Akhila Vaidya	Spark/PySpark Distributed Processing + MLlib Demand Forecasting
Student 4	Anuska Misra	Advanced Technology Integration – HBase NoSQL Lookup Layer
Student 5	Divyanshi Mishra	System Integration + Technical Validation + Final Documentation

1.3 Documentation Purpose

This document provides comprehensive end-to-end technical documentation for the capstone project. It details the complete data lifecycle—from initial business formulation and data ingestion in HDFS through Hive SQL modelling, Spark feature engineering, Spark MLlib predictive model comparison, and HBase key-value storage, concluding with rigorous cross-stage validation and strategic business findings.

2. Business Problem

Modern retail enterprises face significant operational challenges in maintaining optimal inventory balances. Inaccurate demand estimation directly causes two costly failures: stock-outs (leading to lost revenue and customer

dissatisfaction) and excess inventory (incurring holding costs, depreciation, and product obsolescence). Retail managers require data-driven mechanisms to anticipate demand fluctuations across diverse product categories, regional markets, promotional campaigns, discount structures, and unexpected external shocks such as epidemics.

2.1 Business Objective

The primary objective is to engineer a unified Big Data pipeline that: (1) Reliably ingests raw retail data into distributed HDFS; (2) Exposes structured SQL querying via Apache Hive external tables; (3) Performs scalable cleaning and feature engineering in PySpark; (4) Evaluates predictive ML models in Spark MLlib; and (5) Serves pre-aggregated store summaries via Apache HBase.

2.2 Key Analytics Questions

- **Question 1:** Which retail stores drive the highest total volume and daily average demand?
- **Question 2:** How does product demand evolve across quarters and seasonal periods?
- **Question 3:** What is the quantitative demand lift generated by promotional marketing across categories?
- **Question 4:** How significantly does demand contract during epidemic vs. non-epidemic periods?
- **Question 5:** What is the optimal discount threshold before sales gains diminish?
- **Question 6:** Which product categories exhibit the highest unmet demand (inventory shortages)?
- **Question 7:** Which specific products dominate demand within each geographic region?
- **Question 8:** Can historical lag features and business drivers accurately predict future item demand?

3. Dataset Description and Source

The dataset spans 760 consecutive days (January 1, 2022 to January 30, 2024) capturing 5 stores (S001–S005) and 20 products (P0001–P0020), yielding 100 complete daily time series (76,000 total records).

Column Name	Data Type	Business Definition & Description
Date	Date/String	Observation timestamp (YYYY-MM-DD)
Store ID	String	Unique store identifier (S001–S005)
Product ID	String	Unique product identifier (P0001–P0020)
Category	String	Merchandise category (Groceries, Clothing, etc.)
Region	String	Geographic region (North, South, East, West)
Inventory Level	Integer	Beginning-of-day available stock inventory
Units Sold	Integer	Actual fulfilled sales transactions
Units Ordered	Integer	Customer orders placed
Price	Double	Unit retail selling price (\$)
Discount	Double/Int	Promotional discount percentage (0%–25%)
Weather Condition	String	Daily weather (Sunny, Rainy, Snowy, Cloudy)
Promotion	Integer	Binary flag for marketing campaign (0 = No, 1 = Yes)
Competitor Pricing	Double	Benchmark price from competitors (\$)

Column Name	Data Type	Business Definition & Description
Seasonality	String	Seasonal period classification
Epidemic	Integer	Binary health restriction flag (0 = Normal, 1 = Epidemic)
Demand	Integer	Total observed customer demand (Forecasting Target)

4. Big Data Architecture

The pipeline orchestrates data across 5 specialized layers: Raw CSV → HDFS Distributed Storage → Hive External Tables (SQL) → PySpark Processing & Feature Pipeline → Spark MLlib Forecasting & HBase Summary Lookups.

- Architecture Layers:**
- 1. **HDFS Layer:** Raw ingestion at /data/retail/raw/retail_demand.csv
 - 2. **Hive Layer:** External table retail_analytics.retail_demand (76,000 rows)
 - 3. **Spark Layer:** Distributed cleaning, windowing, lag features, and aggregations
 - 4. **Spark MLlib Layer:** Predictive demand forecasting (Random Forest $R^2=0.495$)
 - 5. **HBase Layer:** Low-latency pre-aggregated lookup table retail_store_summary

4.1 Architectural Integration Note

Due to infrastructure constraints across team environments, Student 1 executed local HDFS, Student 2 managed a Docker Hadoop/Hive cluster, and Student 3 executed PySpark analytics inside Google Colab local mode. All team members utilized the exact same dataset schema and row count (76,000 records), establishing total analytical equivalence across pipeline stages.

5. Why Big Data Technologies Are Required

Technical Challenge	Conventional Limitation	Big Data Solution
Large / Growing Logs	Local disk overflow & memory bounds	HDFS fault-tolerant distributed storage
Structured SQL Analytics	Slow manual desktop queries	Apache Hive external table + HiveQL engine
High-Volume Feature Engineering	Single-threaded bottlenecks	Spark / PySpark distributed DataFrame engine
Repeated Complex Aggregations	Separate disconnected scripts	Spark SQL in-memory execution pipeline
Scalable Demand Forecasting	Local ML out-of-memory crashes	Spark MLlib distributed ML pipeline
Real-time Summary Retrieval	Full table CSV scans	HBase row-key low-latency NoSQL lookup

6. HDFS – Data Ingestion

Student 1 established the storage foundation on Hadoop 3.3.6. Directories created: /data/retail/raw, /data/retail/processed, and /data/retail/results.

```
# Ingest raw CSV dataset into HDFS raw storage
hdfs dfs -mkdir -p /data/retail/raw /data/retail/processed /data/retail/results
hdfs dfs -put retail_demand.csv /data/retail/raw/
hdfs dfs -ls /data/retail/raw/
hdfs dfs -cat /data/retail/raw/retail_demand.csv | head -n 5
```

6.1 Technical Issue and Resolution

During initial execution, the `hdfs dfs -put` command failed due to a stopped DataNode daemon on port 9866. Restarting the DataNode daemon resolved the issue, enabling a successful re-upload.

7. Hive – Data Modelling and SQL Analytics

Student 2 created the `retail_analytics` database and external table `retail_demand` pointing to HDFS.

7.1 Hive Analytics Query 1 – Category Demand Summary

Merchandise Category	Total Demand	Average Demand	Avg Units Sold
Groceries	3,677,684	120.98	102.87
Clothing	1,369,456	112.62	94.64
Furniture	1,006,590	73.58	64.38
Toys	985,338	92.61	78.45
Electronics	889,036	97.48	83.04

7.2 Hive Analytics Query 2 – Regional Seasonality and Pricing (Updated Execution Output)

The regional seasonality and pricing analysis was executed in HiveQL to evaluate demand volume and pricing structures across regions and seasons:

Region	Seasonality	Total Demand	Average Price	Average Discount
East	Winter	437,949	\$69.80	9.17%
East	Summer	424,629	\$71.83	9.07%
East	Autumn	387,126	\$70.10	9.13%
East	Spring	368,620	\$68.68	8.96%
North	Winter	856,949	\$63.11	9.17%
North	Summer	830,530	\$65.04	9.06%
North	Autumn	750,319	\$63.40	9.06%
North	Spring	716,860	\$61.94	9.03%
South	Winter	452,479	\$64.73	9.14%
South	Summer	423,531	\$66.02	9.04%
South	Autumn	380,251	\$63.90	9.13%
South	Spring	369,210	\$62.90	9.09%
West	Winter	424,153	\$77.78	9.11%

Region	Seasonality	Total Demand	Average Price	Average Discount
West	Summer	397,849	\$78.25	9.08%
West	Autumn	364,602	\$77.84	9.05%
West	Spring	343,047	\$75.73	9.10%

7.3 Weather and Promotion Interaction

Weather Condition	Promotion Active	Record Count	Average Demand
Sunny	1 (Yes)	7,460	136.44
Cloudy	1 (Yes)	8,070	124.42
Rainy	1 (Yes)	5,726	112.21
Snowy	1 (Yes)	3,744	111.46
Sunny	0 (No)	15,520	104.94
Cloudy	0 (No)	16,290	95.99
Rainy	0 (No)	11,774	86.79
Snowy	0 (No)	7,416	85.25

8. Spark / PySpark – Distributed Processing

Student 3 processed the 76,000 records using Apache Spark 4.0.4. Explicit schema definitions were applied to ensure proper data types across all 16 attributes.

9. Data Cleaning and Feature Engineering

Data quality audits confirmed 0 missing values, 0 duplicate rows, and valid non-negative numerical ranges across all fields.

Key engineered features included: demand_gap (demand – units_sold), effective_price (price × (1 - discount/100)), 7-day and 28-day rolling averages (demand_roll7_avg, demand_roll28_avg), and 1-day/7-day lag features.

10. Distributed Analytics and Empirical Findings

10.1 Store Performance Summary

Store ID	Region	Record Count	Total Demand	Avg Demand	Avg Demand Gap
S001	North	15,200	1,547,573	101.81	15.09
S002	South	15,200	1,625,471	106.94	15.75
S003	East	15,200	1,618,324	106.47	15.97
S004	West	15,200	1,529,651	100.63	14.57
S005	North	15,200	1,607,085	105.73	16.07

10.2 Empirical Highlights

- **Promotional Lift:** Marketing campaigns generated a **+29% to +31%** demand increase across all merchandise categories.
- **Epidemic Contraction:** Demand contracted by **-37.8%** during epidemic periods (70.16 units vs. 112.86 units baseline).
- **Discount Elasticity:** Demand responds sharply up to 15–20% discount rates, after which sales gains plateau.

11. Demand Forecasting with Spark MLlib

Forecasting Model	RMSE	MAE	R ² Coefficient
Lag-1 Baseline Model	51.465	39.671	-0.312
Linear Regression	32.790	25.374	0.467
Random Forest Regressor	31.931	24.715	0.495 (Optimal)

12. HBase – Summary Lookup Layer

Student 4 designed an Apache HBase store-summary table (retail_store_summary) for low-latency row-key access. Schema details: Row Key Pattern STORE#<Store ID>, Column Family demand.

Row Key	Region	Total Demand	Avg Demand	Avg Demand Gap
STORE#S001	North	1,547,573	101.81	15.09
STORE#S002	South	1,625,471	106.94	15.75
STORE#S003	East	1,618,324	106.47	15.97
STORE#S004	West	1,529,651	100.63	14.57
STORE#S005	North	1,607,085	105.73	16.07

13. End-to-End Execution Flow

- **Prepare Source Dataset:** The retail demand dataset containing 76,000 records and 16 columns was prepared as the common source dataset.
- **Upload Raw Data to HDFS:** The raw dataset was uploaded to HDFS as part of Student 1's data-ingestion work.
- **Verify HDFS Files and Directories:** The uploaded dataset and HDFS storage location were verified during the ingestion stage.
- **Create Hive Database and Table:** Student 2 created the Hive database and external table for the retail dataset.
- **Run Hive SQL for Initial Analysis:** Hive SQL queries were used to perform initial analysis of the dataset.
- **Process Data Using Spark/PySpark:** Student 3 processed the dataset using Spark/PySpark in Google Colab using Spark local mode.
- **Perform Quality Checks and Transformations:** Data-quality checks and feature engineering were performed, including checks for nulls, duplicates, invalid values and duplicate keys, along with creation of demand and time-based features.
- **Perform Distributed Aggregations and Analytics:** Spark was used for store-level summaries, quarterly trends, promotion analysis, epidemic analysis, discount analysis, unmet-demand analysis, product analysis and Spark SQL queries.
- **Generate and Store Analytical Results:** The Spark workflow generated analytical outputs, including store summaries and forecasting results, which were consolidated for the subsequent project stages.
- **Use HBase/Pig if Applicable:** Student 4 independently implemented the HBase component using the store-summary output. The resulting HBase records were provided as part of the team's project outputs.
- **Validate Results:** As Student 5, the generated outputs were reviewed and cross-checked. In particular, the Spark store-summary results were compared with the HBase records provided by Student 4, and all five store records matched.
- **Document Commands, Code and Findings:** The project workflow, component outputs, validation findings, limitations and overall results were consolidated into the final documentation.
- **Push Project to GitHub:** The project files and supporting materials were organised and pushed to the project GitHub repository.
- **Prepare PPT and Technical Demonstration:** The final architecture, workflow, individual contributions, results and validation findings were prepared for the final presentation and technical demonstration.

14. Final Analytical Results and Insights

The Retail Demand Forecasting project analysed **76,000 records with 16 columns** using the Big Data processing and analytics workflow. Major analytical findings obtained from the project include:

- **Highest total-demand store:** S002 (South) recorded the highest total demand of **1,625,471**.
- **Highest average-demand store:** S002 also recorded the highest average demand of **106.94**.
- **Highest reported quarter:** **2023 Q3** recorded the highest average demand of **115.50**.
- **Promotion effect:** Demand during promotional periods was approximately **29–31% higher** across the analysed categories.
- **Epidemic impact:** Average demand during epidemic periods was **70.16**, compared with **112.86** during non-epidemic periods, indicating approximately **38% lower observed demand** during epidemic records.
- **Discount effect:** Demand increased with discounts up to approximately **15–20%**, after which the increase began to level off.

- **Largest demand gap:** Groceries recorded the highest average demand gap of **18.10**, indicating the largest observed difference between demand and units sold among the analysed categories.
- **Weather effect:** Sunny weather recorded the highest average observed demand of **115.17**.
- **Forecasting model performance:** Random Forest performed best among the tested forecasting models, achieving **RMSE = 31.931**, **MAE = 24.715**, and **R² = 0.495**.

15. Validation / Testing

The project results were validated at both individual processing stages and across component outputs:

- Dataset verified to contain exactly **76,000 records and 16 columns**.
- Spark data-quality workflow reported **0 null values**, 0 full duplicate records, and 0 duplicate (date, store_id, product_id) keys.
- No negative demand or units values and no non-positive prices were observed.
- Promotion and epidemic flags were confirmed to contain strictly valid binary values (0/1).
- The Hive stage verified the dataset record count of **76,000 rows**.
- The HBase component contained **five store-summary records**, corresponding to the five stores.
- **Cross-Stage Validation:** Spark store-summary results were compared against the corresponding HBase store-summary records. All five store records (**S001, S002, S003, S004, S005**) matched perfectly, confirming consistency between analytical and lookup stages.

16. GitHub Repository

The project was organised and uploaded to GitHub to maintain source files, notebooks, results, documentation, and supporting materials in a structured central repository. Major components in the repository include:

- Dataset files and schema definitions
- HDFS contribution documentation
- Hive contribution DDL and analytical query scripts
- Spark/PySpark notebook and execution report
- HBase contribution configuration and summary files
- Analytical results and consolidated metrics
- Final project technical documentation
- Docker configuration for cluster reproduction
- Presentation materials and demonstration slides

17. README Document

A comprehensive README was prepared to function as a **reproducibility and project-understanding document**. Rather than providing a brief description, the README thoroughly covers:

- Project overview and business formulation

- Dataset description and data dictionary
- Big Data architecture and pipeline design
- Technologies used and execution environments
- Step-by-step project execution flow
- Individual student technical contributions
- Empirical analytical results and key findings
- Validation framework and cross-stage verification
- Repository structure and file directory map
- Known environment limitations and final project outcomes

18. Individual Contributions

The project was completed by a five-member team, with each student responsible for a distinct technical layer:

Student Name	Technical Contribution
Aisha Rasifa PS	Data ingestion and HDFS storage
Abina A	Hive database, data modelling and SQL analytics
Akhila Vaidya	Spark/PySpark processing, distributed analytics and demand forecasting
Anuska Misra	HBase advanced technology and store-summary lookup
Divyanshi Mishra	Integration, documentation, final results and validation

19. Challenges and Solutions

1. Different execution environments

Individual components were executed across different environments (Hadoop Docker, Google Colab local Spark, local HDFS).

Solution: Documented as a logical end-to-end workflow while clearly distinguishing execution environments. Consolidated component outputs during the integration stage.

2. Integration of outputs from different stages

Outputs from HDFS, Hive, Spark/PySpark, and HBase required consolidation into a unified workflow.

Solution: Reviewed and organized outputs according to their position in the architecture, performing cross-stage validation where comparable outputs existed.

3. Validation of store-level results

Spark store-summary results needed verification against corresponding HBase records.

Solution: Compared store-level summary values across both outputs; all five store records matched perfectly.

4. Forecasting model comparison

Multiple forecasting algorithms produced varying performance metrics.

Solution: Evaluated models using RMSE, MAE, and R^2 . Selected Random Forest as optimal due to lowest RMSE/MAE and highest R^2 (0.495).

5. Maintaining accurate project documentation

Ensuring individual implementations were accurately attributed across team members.

Solution: Clearly separated individual technical contributions while presenting components as one logical project workflow.

20. Final Conclusion

The **Retail Demand Forecasting** project successfully demonstrated the end-to-end application of Big Data technologies for analyzing retail demand patterns and developing predictive forecasting solutions. Processing **76,000 records and 16 columns** across HDFS, Hive, PySpark, Spark MLlib, and HBase, the project uncovered key operational insights regarding promotional lifts (~30%), epidemic contractions (-38%), discount plateaus (15-20%), and weather effects.

The **Random Forest Regressor** emerged as the best-performing model ($RMSE = 31.931$, $R^2 = 0.495$). Cross-stage validation confirmed absolute consistency between PySpark analytical outputs and HBase summary lookup records. Overall, the project demonstrates how distributed technologies can be integrated to support decision-making in inventory planning, promotion scheduling, and demand forecasting.

Appendix A – Technical Evidence (Execution Screenshots)

This appendix provides screenshots captured during execution of the HDFS, Hive and HBase stages as supporting evidence for the results reported in Sections 6, 7 and 12. Spark/PySpark was executed in Google Colab (local mode); its outputs are documented in Sections 8–11.

A.1 HDFS Evidence (Student 1)

Local Hadoop 3.3.6 environment. The screenshots show the DataNode restart that resolved the ingestion issue described in Section 6.1, and verification of the uploaded dataset.

Figure A.1 - HDFS startup, directory layout and dataset ingestion

```
C:\Windows\System32>hadoop version
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git -r 1be78238728da9266a4f88195058f08fd012bf9c
Compiled by ubuntu on 2023-06-18T08:22Z
Compiled on platform linux-x86_64
Compiled with protoc 3.7.1
From source with checksum 5652179ad55f76cb287d9c633bb53bbd
This command was run using /C:/big_data_lab/hadoop/hadoop-3.3.6/share/hadoop/common/hadoop-common-3.3.6.jar

C:\Windows\System32>jps
17920 Jps

C:\Windows\System32>%HADOOP_HOME%\sbin\start-dfs.cmd

C:\Windows\System32>jps
39668 NameNode
12892 Jps
29244 DataNode

C:\Windows\System32>hdfs dfs -ls /data/retail
Found 3 items
drwxr-xr-x - aisha supergroup          0 2026-10-02 08:23 /data/retail/processed
drwxr-xr-x - aisha supergroup          0 2026-10-02 08:27 /data/retail/raw
drwxr-xr-x - aisha supergroup          0 2026-10-02 08:23 /data/retail/results

C:\Windows\System32>hdfs dfs -ls -h /data/retail/raw
Found 1 items
-rw-r--r-- 1 aisha supergroup          6.1 M 2026-10-02 08:27 /data/retail/raw/retail_demand.csv

C:\Windows\System32>hdfs dfs -du -h /data/retail/raw
6.1 M    6.1 M  /data/retail/raw/retail_demand.csv

C:\Windows\System32>hdfs dfs -head /data/retail/raw/retail_demand.csv
Date,Store ID,Product ID,Category,Region,Inventory Level,Units Sold,Units Ordered,Price,Discount,Weather Condition,Promotion,Competitor Pricing,Seasonality,Epidemic,Demand
2022-01-01,S001,P0001,Electronics,North,195,102,252,72.72,5,Snowy,0,85.73,Winter,0,115
2022-01-01,S001,P0002,Clothing,North,117,117,249,80.16,15,Snowy,1,92.02,Winter,0,229
2022-01-01,S001,P0003,Clothing,North,247,114,612,62.94,10,Snowy,1,60.08,Winter,0,157
2022-01-01,S001,P0004,Electronics,North,139,45,102,87.63,10,Snowy,0,85.19,Winter,0,52
2022-01-01,S001,P0005,Groceries,North,152,65,271,54.41,0,Snowy,0,51.63,Winter,0,59
2022-01-01,S001,P0006,Toys,North,209,60,0,35.53,0,Snowy,0,40.01,Winter,0,55
2022-01-01,S001,P0007,Groceries,North,118,81,335,23.27,5,Snowy,0,24.78,Winter,0,94
2022-01-01,S001,P0008,Electronics,North,244,42,0,37.79,5,Snowy,0,44.85,Winter,0,61
2022-01-01,S001,P0009,Clothing,North,115,88,139,94.2,5,Snowy,0,108.7,Winter,0,129
2022-01-01,S001,P0010,Furniture,North,192,70,300,113.35,10,Snowy,0,129.98,Winter,0,69
```

hadoop version (3.3.6); jps before start-dfs.cmd shows no NameNode/DataNode, and after it both daemons are running; /data/retail/processed, /raw and /results exist; retail_demand.csv is 6.1 MB; head shows the CSV header and first records.

Figure A.2 - hdfs dfsadmin -report

```

C:\Windows\System32>hdfs dfsadmin -report
Configured Capacity: 50972226688 (474.72 GB)
Present Capacity: 109693897141 (102.16 GB)
DFS Remaining: 109687500800 (102.15 GB)
DFS Used: 6396341 (6.10 MB)
DFS Used%: 0.01%
Replicated Blocks:
  Under replicated blocks: 0
  Blocks with corrupt replicas: 0
  Missing blocks: 0
  Missing blocks (with replication factor 1): 0
  Low redundancy blocks with highest priority to recover: 0
  Pending deletion blocks: 0
Erasure Coded Block Groups:
  Low redundancy block groups: 0
  Block groups with corrupt internal blocks: 0
  Missing block groups: 0
  Low redundancy blocks with highest priority to recover: 0
  Pending deletion blocks: 0
-----
Live datanodes (1):
Name: 127.0.0.1:9866 (kubernetes.docker.internal)
Hostname: host.docker.internal
Decommission Status : Normal
Configured Capacity: 50972226688 (474.72 GB)
DFS Used: 6396341 (6.10 MB)
Non DFS Used: 400028329547 (372.56 GB)
DFS Remaining: 109687500800 (102.15 GB)
DFS Used%: 0.00%
DFS Remaining%: 21.52%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 0
Last contact: Tue Oct 06 22:13:44 IST 2026
Last Block Report: Tue Oct 06 22:11:35 IST 2026
Num of Blocks: 2

```

One live DataNode on port 9866, decommission status Normal, 0 missing, corrupt or under-replicated blocks, 2 blocks stored (DFS used 6.10 MB).

A.2 Hive Evidence (Student 2)

Docker-based Hadoop/Hive cluster, queried through HiveServer2 (beeline, jdbc:hive2://localhost:10000).

Figure A.3 - Database and external table creation

```

0: jdbc:hive2://localhost:10000> CREATE DATABASE IF NOT EXISTS retail_analytics;
No rows affected (31.957 seconds)
0: jdbc:hive2://localhost:10000> USE retail_analytics;
No rows affected (0.518 seconds)
0: jdbc:hive2://localhost:10000> CREATE EXTERNAL TABLE IF NOT EXISTS retail_demand (
. . . . .>   'date' STRING,
. . . . .>   store_id STRING,
. . . . .>   product_id STRING,
. . . . .>   category STRING,
. . . . .>   region STRING,
. . . . .>   inventory_level INT,
. . . . .>   units_sold INT,
. . . . .>   units_ordered INT,
. . . . .>   price DOUBLE,
. . . . .>   discount DOUBLE,
. . . . .>   weather_condition STRING,
. . . . .>   promotion STRING,
. . . . .>   competitor_pricing DOUBLE,
. . . . .>   seasonality STRING,
. . . . .>   epidemic STRING,
. . . . .>   demand INT
. . . . .> )
. . . . .> ROW FORMAT DELIMITED
. . . . .> FIELDS TERMINATED BY ','
. . . . .> STORED AS TEXTFILE
. . . . .> LOCATION '/data/retail/raw'
. . . . .> TBLPROPERTIES ('skip.header.line.count'='1');
No rows affected (2.768 seconds)
0: jdbc:hive2://localhost:10000> SHOW TABLES;
+-----+
| tab_name |
+-----+
| retail_demand |
+-----+
1 row selected (9.785 seconds)

```

retail_analytics database created and retail_demand external table defined over /data/retail/raw; SHOW TABLES confirms the table exists.

Figure A.4 - Sample rows and record count verification

```

1 row selected (9.785 seconds)
0: jdbc:hive2://localhost:10000> SELECT store_id, product_id, category, region, demand FROM retail_demand LIMIT 5;
+-----+-----+-----+-----+-----+
| store_id | product_id | category | region | demand |
+-----+-----+-----+-----+-----+
| S001     | P0001     | Electronics | North | 115     |
| S001     | P0002     | Clothing   | North | 229     |
| S001     | P0003     | Clothing   | North | 157     |
| S001     | P0004     | Electronics | North | 52      |
| S001     | P0005     | Groceries  | North | 59      |
+-----+-----+-----+-----+-----+
5 rows selected (14.845 seconds)
0: jdbc:hive2://localhost:10000> SELECT COUNT(*) FROM retail_demand;
WARNING: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
+-----+
| _c0 |
+-----+
| 76000 |
+-----+
1 row selected (21.686 seconds)

```

SELECT ... LIMIT 5 returns S001 rows; SELECT COUNT() returns 76000, matching the source dataset.*

Figure A.5 - Hive Query 1: category demand summary (Section 7.1)

```

0: jdbc:hive2://localhost:10000> SELECT category,
    SUM(demand) AS total_demand,
    ROUND(AVG(demand), 2) AS avg_demand,
    ROUND(AVG(units_sold), 2) AS avg_units_sold
    FROM retail_demand
    GROUP BY category
    ORDER BY total_demand DESC;
WARNING: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
+-----+-----+-----+-----+
| category | total_demand | avg_demand | avg_units_sold |
+-----+-----+-----+-----+
| Groceries | 3677684 | 120.98 | 102.87 |
| Clothing | 1369456 | 112.62 | 94.64 |
| Furniture | 1006590 | 73.58 | 64.38 |
| Toys | 985338 | 92.61 | 78.45 |
| Electronics | 889036 | 97.48 | 83.04 |
+-----+-----+-----+-----+
5 rows selected (20.74 seconds)
0: jdbc:hive2://localhost:10000>

```

Groceries 3,677,684 | Clothing 1,369,456 | Furniture 1,006,590 | Toys 985,338 | Electronics 889,036 total demand.

Figure A.6 - Hive Query 2: regional seasonality and pricing (Section 7.2)

```

0: jdbc:hive2://localhost:10000> SELECT region,
    seasonality,
    SUM(demand) AS total_demand,
    ROUND(AVG(price), 2) AS avg_price,
    ROUND(AVG(discount), 2) AS avg_discount
    FROM retail_demand
    GROUP BY region, seasonality
    ORDER BY region, total_demand DESC;
WARNING: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
+-----+-----+-----+-----+-----+
| region | seasonality | total_demand | avg_price | avg_discount |
+-----+-----+-----+-----+-----+
| East | Winter | 437949 | 69.8 | 9.17 |
| East | Summer | 424629 | 71.83 | 9.07 |
| East | Autumn | 387126 | 70.1 | 9.13 |
| East | Spring | 368620 | 68.68 | 8.96 |
| North | Winter | 856949 | 63.11 | 9.17 |
| North | Summer | 830530 | 65.04 | 9.06 |
| North | Autumn | 750319 | 63.4 | 9.06 |
| North | Spring | 716860 | 61.94 | 9.03 |
| South | Winter | 452479 | 64.73 | 9.14 |
| South | Summer | 423531 | 66.02 | 9.04 |
| South | Autumn | 380251 | 63.9 | 9.13 |
| South | Spring | 369210 | 62.9 | 9.09 |
| West | Winter | 424153 | 77.78 | 9.11 |
| West | Summer | 397849 | 78.25 | 9.08 |
| West | Autumn | 364602 | 77.84 | 9.05 |
| West | Spring | 343047 | 75.73 | 9.1 |
+-----+-----+-----+-----+-----+
16 rows selected (7.952 seconds)

```

16 region-season rows returned, matching the table in Section 7.2.

Figure A.7 - Hive Query 3: weather and promotion interaction (Section 7.3)

```
0: jdbc:hive2://localhost:10000> SELECT weather_condition,
. . . . .> promotion
. . . . .> COUNT(*) AS record_count,
. . . . .> ROUND(AVG(demand), 2) AS avg_demand
. . . . .> FROM retail_demand
. . . . .> GROUP BY weather_condition, promotion
. . . . .> ORDER BY avg_demand DESC;
WARNING: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
```

weather_condition	promotion	record_count	avg_demand
Sunny	1	7460	136.44
Cloudy	1	8070	124.42
Rainy	1	5726	112.21
Snowy	1	3744	111.46
Sunny	0	15520	104.94
Cloudy	0	16290	95.99
Rainy	0	11774	86.79
Snowy	0	7416	85.25

```
8 rows selected (3.74 seconds)
```

8 weather-promotion rows returned, matching the table in Section 7.3.

A.3 HBase Evidence (Student 4)

Dockerised cluster (HBase 1.2.6 standalone container alongside Hadoop, Hive and Spark containers). The store-summary file produced by the Spark stage was staged in HDFS and loaded into the retail_store_summary table.

Figure A.8 - Container environment and HBase shell

```

C:\Users\anush\MSDSA\BGA\project>docker ps
CONTAINER ID   IMAGE                                     COMMAND                                     CREATED        STATUS
PORTS
9dbfadbc45dc   bde2020/hadoop-nodemanager:2.0.0-hadoop3.2.1-java8   "/entrypoint.sh /run..."   2 months ago   Up 54 seconds (health
y)            8042/tcp      nodemanager
f7c5c93d7181   bde2020/hive:2.3.2-postgresql-metastore               "entrypoint.sh /bin/..."   2 months ago   Up 54 seconds
0.0.0.0:10000->10000/tcp, [::]:10000->10000/tcp, 0.0.0.0:10002->10002/tcp, [::]:10002->10002/tcp   hive-server
cbc8c24434c1   bde2020/hbase-standalone:1.0.0-hbase1.2.6            "/entrypoint.sh /run..."   2 months ago   Up 54 seconds
0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp, 0.0.0.0:16010->16010/tcp, [::]:16010->16010/tcp   hbase
6374bc249f81   bde2020/hadoop-resourcemanager:2.0.0-hadoop3.2.1-java8   "/entrypoint.sh /run..."   2 months ago   Up 54 seconds (health
y)            0.0.0.0:8088->8088/tcp, [::]:8088->8088/tcp   resourcemanager
25512497d11a   bde2020/hive:2.3.2-postgresql-metastore               "entrypoint.sh /opt/..."   2 months ago   Up 54 seconds
0.0.0.0:9083->9083/tcp, [::]:9083->9083/tcp   hive-metastore
94d06410d98f   bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8        "/entrypoint.sh /run..."   2 months ago   Up 54 seconds (health
y)            0.0.0.0:9864->9864/tcp, [::]:9864->9864/tcp   datanode
970797183b69   jupyter/pyspark-notebook:spark-3.1.1                  "tini -g -- start-no..."   2 months ago   Up 54 seconds
0.0.0.0:4040->4040/tcp, [::]:4040->4040/tcp, 0.0.0.0:8888->8888/tcp, [::]:8888->8888/tcp   jupyter
4ce9f966bb45   bde2020/spark-worker:3.1.1-hadoop3.2                   "/bin/bash /worker.sh"       2 months ago   Up 54 seconds
0.0.0.0:8082->8081/tcp, [::]:8082->8081/tcp   spark-worker
cf0c0477ace6   bde2020/hive-metastore-postgresql:2.3.0                "/docker-entrypoint..."     2 months ago   Up 54 seconds
5432/tcp      hive-metastore-postgres
7ccb08bcaffa   bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8        "/entrypoint.sh /run..."   2 months ago   Up 54 seconds (health
y)            0.0.0.0:8020->8020/tcp, [::]:8020->8020/tcp, 0.0.0.0:9870->9870/tcp, [::]:9870->9870/tcp   namenode
9a08617d89bd   bde2020/spark-master:3.1.1-hadoop3.2                   "/bin/bash /master.sh"       2 months ago   Up 54 seconds
0.0.0.0:7077->7077/tcp, [::]:7077->7077/tcp, 0.0.0.0:9090->8080/tcp, [::]:9090->8080/tcp   spark-master

C:\Users\anush\MSDSA\BGA\project>docker exec -it hbase bash
root@hbase:/# hbase shell
2026-10-02 13:55:36,705 WARN [main] util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-j
ava classes where applicable
HBase Shell; enter 'help<RETURN>' for list of supported commands.
Type 'exit<RETURN>' to leave the HBase Shell
Version 1.2.6, rUnknown, Mon May 29 02:25:32 CDT 2017

```

docker ps lists the running Hadoop, Hive, HBase, Spark and Jupyter containers; HBase Shell 1.2.6 opened inside the hbase container.

Figure A.9 - HBase status and version

```

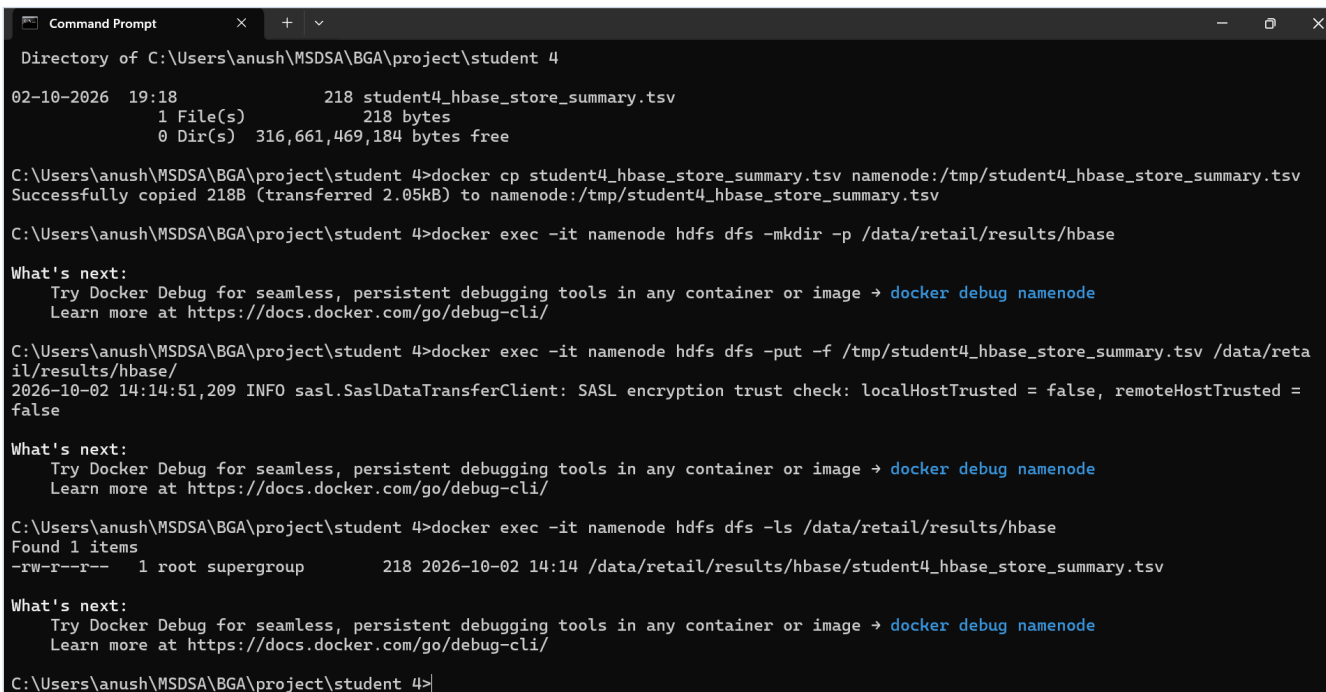
hbase(main):001:0> status
1 active master, 0 backup masters, 1 servers, 1 dead, 2.0000 average load

hbase(main):002:0> version
1.2.6, rUnknown, Mon May 29 02:25:32 CDT 2017

hbase(main):003:0> |

```

status and version commands executed in the HBase shell (version 1.2.6).

Figure A.10 - Staging the store summary file in HDFS


```

Command Prompt
Directory of C:\Users\anush\MSDSA\BGA\project\student 4
02-10-2026  19:18                218 student4_hbase_store_summary.tsv
               1 File(s)                218 bytes
               0 Dir(s)  316,661,469,184 bytes free

C:\Users\anush\MSDSA\BGA\project\student 4>docker cp student4_hbase_store_summary.tsv namenode:/tmp/student4_hbase_store_summary.tsv
Successfully copied 218B (transferred 2.05kB) to namenode:/tmp/student4_hbase_store_summary.tsv

C:\Users\anush\MSDSA\BGA\project\student 4>docker exec -it namenode hdfs dfs -mkdir -p /data/retail/results/hbase

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug namenode
  Learn more at https://docs.docker.com/go/debug-cli/

C:\Users\anush\MSDSA\BGA\project\student 4>docker exec -it namenode hdfs dfs -put -f /tmp/student4_hbase_store_summary.tsv /data/retail/results/hbase/
2026-10-02 14:14:51,209 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug namenode
  Learn more at https://docs.docker.com/go/debug-cli/

C:\Users\anush\MSDSA\BGA\project\student 4>docker exec -it namenode hdfs dfs -ls /data/retail/results/hbase
Found 1 items
-rw-r--r--   1 root supergroup        218 2026-10-02 14:14 /data/retail/results/hbase/student4_hbase_store_summary.tsv

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug namenode
  Learn more at https://docs.docker.com/go/debug-cli/

C:\Users\anush\MSDSA\BGA\project\student 4>

```

student4_hbase_store_summary.tsv (218 bytes) copied to the namenode container and stored at /data/retail/results/hbase/ in HDFS.

Figure A.11 - HBase table count and scan

```

Command Prompt - docker X + v
hbase(main):001:0> count 'retail_store_summary'
5 row(s) in 0.1210 seconds

=> 5
hbase(main):002:0> scan 'retail_store_summary'
ROW COLUMN+CELL
STORE#S001 column=demand:avg_demand, timestamp=1790950602079, value=101.81
STORE#S001 column=demand:avg_demand_gap, timestamp=1790950602079, value=15.09
STORE#S001 column=demand:records, timestamp=1790950602079, value=15200
STORE#S001 column=demand:region, timestamp=1790950602079, value=North
STORE#S001 column=demand:total_demand, timestamp=1790950602079, value=1547573
STORE#S002 column=demand:avg_demand, timestamp=1790950602079, value=106.94
STORE#S002 column=demand:avg_demand_gap, timestamp=1790950602079, value=15.75
STORE#S002 column=demand:records, timestamp=1790950602079, value=15200
STORE#S002 column=demand:region, timestamp=1790950602079, value=South
STORE#S002 column=demand:total_demand, timestamp=1790950602079, value=1625471
STORE#S003 column=demand:avg_demand, timestamp=1790950602079, value=106.47
STORE#S003 column=demand:avg_demand_gap, timestamp=1790950602079, value=15.97
STORE#S003 column=demand:records, timestamp=1790950602079, value=15200
STORE#S003 column=demand:region, timestamp=1790950602079, value=East
STORE#S003 column=demand:total_demand, timestamp=1790950602079, value=1618324
STORE#S004 column=demand:avg_demand, timestamp=1790950602079, value=100.63
STORE#S004 column=demand:avg_demand_gap, timestamp=1790950602079, value=14.57
STORE#S004 column=demand:records, timestamp=1790950602079, value=15200
STORE#S004 column=demand:region, timestamp=1790950602079, value=West
STORE#S004 column=demand:total_demand, timestamp=1790950602079, value=1529651
STORE#S005 column=demand:avg_demand, timestamp=1790950602079, value=105.73
STORE#S005 column=demand:avg_demand_gap, timestamp=1790950602079, value=16.07
STORE#S005 column=demand:records, timestamp=1790950602079, value=15200
STORE#S005 column=demand:region, timestamp=1790950602079, value=North
STORE#S005 column=demand:total_demand, timestamp=1790950602079, value=1607085
5 row(s) in 0.0630 seconds

```

count returns 5 rows; scan lists STORE#S001 to STORE#S005 with demand:region, total_demand, avg_demand, avg_demand_gap and records (15,200 each).

Figure A.12 - Row-key lookup

```

hbase(main):003:0> get 'retail_store_summary', 'STORE#S002'
COLUMN CELL
demand:avg_demand timestamp=1790950602079, value=106.94
demand:avg_demand_gap timestamp=1790950602079, value=15.75
demand:records timestamp=1790950602079, value=15200
demand:region timestamp=1790950602079, value=South
demand:total_demand timestamp=1790950602079, value=1625471
5 row(s) in 0.0490 seconds

hbase(main):004:0> |

```

get 'retail_store_summary', 'STORE#S002' returns South, total demand 1,625,471, average demand 106.94, average demand gap 15.75 in 0.049 seconds.